

Arquitectura Distribuida

1. Síntesis de decisiones previas

A partir del análisis realizado en los hitos anteriores, se definió una arquitectura de persistencia polígloa para responder a las distintas necesidades de datos del Fixture 2030. La selección de cada tecnología se realizó considerando principalmente el volumen esperado, la velocidad de generación, el patrón de acceso, la consistencia necesaria y las posibilidades de escalabilidad de cada subsistema.

Las decisiones de persistencia vigentes son las siguientes:

Necesidad de datos	Tecnología seleccionada	Responsabilidad principal
Logs generados durante los partidos	Cassandra	Registrar el alto volumen de información generado continuamente durante los encuentros
Partidos y valores en tiempo real	Redis	Mantener el estado vigente de los partidos con baja latencia
Partidos finalizados y datos estáticos	MongoDB	Persistir partidos finalizados, jugadores, selecciones y estadios
Grupos y puntajes	IRIS	Mantener la composición de los grupos, puntajes y posiciones de manera consistente
Predicciones de usuarios	Neo4j	Representar las relaciones entre usuarios, partidos y pronósticos
Métricas y estadísticas históricas	InfluxDB	Almacenar información temporal y permitir el análisis de su evolución

Estas decisiones mantienen el mapa de persistencia definido en el Hito 2, donde Redis administra el estado transitorio de los partidos y posteriormente la información definitiva pasa a MongoDB, mientras que Cassandra recibe los logs generados por los eventos. IRIS, Neo4j e InfluxDB mantienen responsabilidades independientes para grupos y puntajes, predicciones y datos temporales respectivamente.

El escenario del Fixture 2030 exige que estas decisiones sean llevadas a una arquitectura distribuida. La plataforma deberá operar en múltiples regiones geográficas, atender entre 2 y 3 millones de usuarios simultáneos, superar las 100.000 solicitudes por segundo en situaciones

de alta demanda, mantener una latencia inferior a 100 ms para el 95 % de las consultas críticas y alcanzar una disponibilidad del 99,99 %.

Para profundizar el análisis distribuido se priorizan cuatro subsistemas:

1. logs de partidos con Cassandra
2. partidos en tiempo real con Redis
3. grupos y puntajes con IRIS
4. métricas históricas con InfluxDB

La selección permite estudiar necesidades diferentes: escritura masiva y continua, baja latencia durante una alta concurrencia, integridad de información sensible para el usuario y crecimiento sostenido de datos temporales.

MongoDB y Neo4j continúan formando parte de la arquitectura general, pero no serán analizados con el mismo nivel de profundidad en las siguientes secciones. La prioridad se coloca sobre los cuatro componentes donde las decisiones de consistencia, disponibilidad, replicación y distribución tienen mayor impacto sobre el comportamiento global de la plataforma.

2. Análisis CAP y modelos de consistencia

Al operar en múltiples regiones, la plataforma debe contemplar posibles particiones de red. Ante ese escenario, cada subsistema debe decidir si resulta más importante mantener la consistencia de los datos o continuar disponible aunque exista información temporalmente desactualizada. La decisión varía según el impacto funcional de cada dato.

Subsistema	Prioridad durante una partición	Modelo de consistencia	Impacto aceptado
Logs de partidos (Cassandra)	Disponibilidad y tolerancia a particiones	Consistencia eventual	La generación de logs no debe detenerse. Se acepta una diferencia temporal entre réplicas mientras posteriormente queden igualadas.
Partidos en tiempo real (Redis)	Consistencia y disponibilidad	Consistencia fuerte	Un marcador o evento no debería mostrarse de manera diferente
Grupos y puntajes (IRIS)	Consistencia y tolerancia a particiones	Consistencia fuerte	Los integrantes de un mismo grupo deben observar los mismos puntajes, posiciones y composición.
Métricas y estadísticas	Disponibilidad y tolerancia a particiones	Consistencia eventual	Un retraso breve en la propagación de una métrica es

históricas (InfluxDB)			aceptable si permite mantener la ingesta continua.
--------------------------	--	--	--

3. Replicación, particionamiento y quórum

La replicación busca mantener el servicio ante la pérdida de un nodo, mientras que el particionamiento distribuye los datos y la carga entre diferentes nodos. La estrategia se define según el patrón de acceso de cada subsistema y debe evitar que una única partición concentre una cantidad desproporcionada de operaciones.

Subsistema	Replicación y topología	Criterio de particionamiento	Trade-off
Logs de partidos (Cassandra)	Múltiples réplicas con topología peer-to-peer, distribuidas entre regiones	Hash utilizando partido_id junto con una división temporal	Favorece escrituras continuas y tolerancia a fallos. La división temporal reduce el riesgo de que un partido concentre todas las escrituras en una única partición, a cambio de distribuir sus logs entre varias particiones.
Partidos en tiempo real (Redis)	Nodo activo con réplica de respaldo (Active-Passive)	Hash por partido_id	Permite localizar rápidamente el estado de cada partido y recuperar el servicio ante una falla. Un encuentro de alta audiencia puede generar un hotspot al concentrar gran parte del tráfico sobre la misma partición.
Grupos y puntajes (IRIS)	Nodo activo con réplicas sincronizadas	Particionamiento geográfico por región o país del grupo	Reduce la latencia acercando los grupos a sus usuarios y mantiene copias para recuperación. Como riesgo, una región con mayor actividad puede concentrar más carga que las demás.
Métricas y estadísticas históricas (InfluxDB)	Replicación asíncrona entre nodos regionales	Partido y período temporal	Permite distribuir el crecimiento histórico y continuar la ingesta aunque una réplica se retrase. Se acepta una diferencia temporal entre copias para evitar aumentar la latencia de escritura.

El particionamiento por hash favorece una distribución más uniforme, mientras que el particionamiento geográfico reduce la distancia entre los datos y sus usuarios. El principal

riesgo son los hotspots, especialmente en Redis durante un partido popular, por lo que la distribución de carga deberá revisarse cuando una partición concentre una proporción excesiva del tráfico.

Para Cassandra se propone un factor de replicación $N=3$, con $R=1$ y $W=1$ para los logs. De esta manera, una escritura o lectura puede completarse con la respuesta de un único nodo, favoreciendo baja latencia y disponibilidad. Como $R + W \leq N$, no se garantiza consistencia fuerte y se mantiene la consistencia eventual definida anteriormente. El costo aceptado es la posibilidad de realizar temporalmente una lectura sobre una réplica que todavía no recibió la última escritura. Para los demás subsistemas no se aplican parámetros N , R y W de forma explícita, ya que sus garantías se expresan mediante las estrategias de replicación y consistencia definidas en el diseño.

4. Escalabilidad, fallos y métricas

La arquitectura debe poder absorber picos superiores a 100.000 solicitudes por segundo, entre 2 y 3 millones de usuarios simultáneos y mantener una latencia inferior a 100 ms para el 95 % de las consultas críticas. Por este motivo, el crecimiento se plantea principalmente mediante escalamiento horizontal, agregando nodos y redistribuyendo carga cuando un subsistema se aproxima a su límite de capacidad.

Los componentes más sensibles durante un partido serán Redis, por la concentración de consultas sobre el estado en vivo, y Cassandra e InfluxDB, por el aumento de escrituras. IRIS tendrá una carga más asociada a las actualizaciones de puntajes y consultas de grupos. El escalamiento vertical puede utilizarse como mejora puntual, pero frente al crecimiento sostenido se prioriza agregar nodos para evitar depender de la capacidad de un único servidor.

Escenario de fallo	Respuesta esperada
Caída de un nodo	Las réplicas disponibles continúan atendiendo solicitudes. En Redis o IRIS, una réplica podrá asumir el rol activo; Cassandra continúa utilizando los demás nodos peer-to-peer.
Partición o fallo de una región	Cassandra e InfluxDB mantienen la operación disponible aceptando sincronización posterior. Redis e IRIS priorizan evitar estados contradictorios, pudiendo limitar temporalmente determinadas operaciones.
Saturación durante un partido popular	Se distribuye carga incorporando capacidad horizontal y, cuando corresponda, redistribuyendo particiones. Se deberá vigilar especialmente la aparición de hotspots sobre el partido de mayor audiencia.
Retraso entre réplicas	En logs y métricas se tolera mientras las copias converjan posteriormente. En partidos en vivo y puntajes deberá evitarse responder con una réplica desactualizada cuando pueda producir

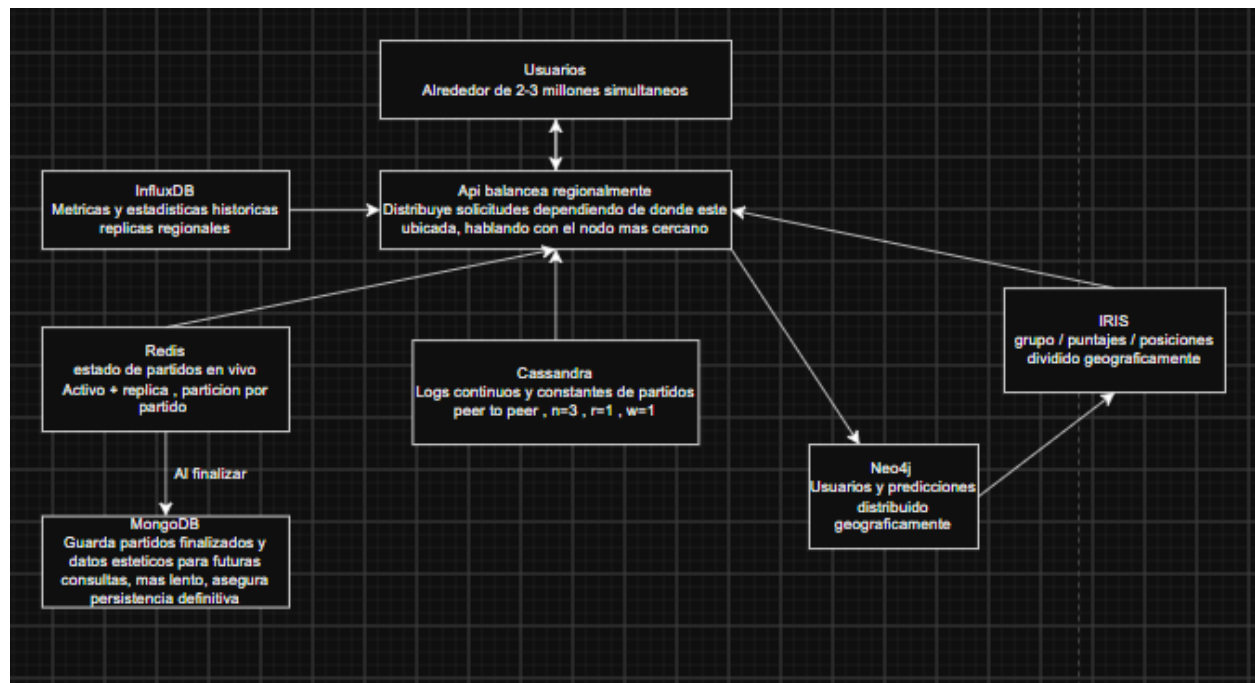
información contradictoria.

Para verificar que la arquitectura cumpla los objetivos planteados se controlarán las siguientes métricas:

- **Disponibilidad:** porcentaje de solicitudes atendidas correctamente, con objetivo de 99,99 %.
- **Latencia:** tiempo de respuesta y percentil 95, buscando mantener las consultas críticas por debajo de 100 ms.
- **Throughput:** cantidad de lecturas y escrituras procesadas por segundo, especialmente durante los picos de los partidos.
- **Utilización:** uso de CPU, memoria y capacidad de almacenamiento por nodo, para detectar anticipadamente saturación.
- **Estado de las réplicas:** cantidad de réplicas disponibles, retraso de sincronización y nodos fuera de servicio.
- **Distribución de carga:** operaciones recibidas por nodo o partición, permitiendo detectar hotspots y desequilibrios.

Estas métricas permiten identificar degradaciones antes de que se traduzcan en una caída del servicio y comprobar posteriormente si las decisiones de replicación, particionamiento y escalabilidad propuestas resultan suficientes para la carga real del Fixture 2030.

5. Diagrama y decisiones pendientes



La arquitectura incorpora réplicas y particiones según las necesidades analizadas: Cassandra distribuye los logs entre nodos peer-to-peer; Redis utiliza un nodo activo con réplica de respaldo; IRIS prioriza réplicas sincronizadas para mantener consistencia; e InfluxDB distribuye la ingesta de métricas entre nodos regionales. El diagrama también debe reflejar el flujo de Redis hacia MongoDB al consolidarse la información del partido y la generación de logs hacia Cassandra.

Quedan pendientes de validación durante los próximos hitos:

- Comprobar si las estrategias de particionamiento evitan hotspots durante encuentros de alta audiencia;
- Validar que la cantidad de réplicas propuesta resulte suficiente ante la caída de un nodo o una región;
- Determinar el mecanismo definitivo y la frecuencia de persistencia de Redis hacia MongoDB;
- Comprobar mediante pruebas de carga si la arquitectura mantiene la latencia objetivo de menos de 100 ms y soporta los picos superiores a 100.000 solicitudes por segundo;
- Revisar el crecimiento del almacenamiento de logs y métricas para definir políticas de conservación;
- Validar que las decisiones de consistencia y replicación respondan como se espera ante fallos reales.

Estas validaciones permitirán ajustar la arquitectura preliminar antes de avanzar hacia su implementación, manteniendo como objetivos principales la disponibilidad del 99,99 %, la baja latencia y la capacidad de operar frente a fallos parciales sin comprometer la integridad de los datos críticos.